

파일 업로드

파일 업로드

- 사용자가 웹 서비스에 파일을 전송하는 과정을 의미
- 이미지, 문서, 오디오 등 다양한 유형의 파일을 포함
- FastAPI는 파일 업로드를 간단하게 처리할 수 있도록 여러 도구와 기능을 제공
 - HTTP: 웹상에서 데이터를 주고받을 수 있는 프로토콜
 - POST 메서드: HTTP 요청 메서드 중 하나로, 서버에 데이터를 보내는 데 사용됨
 - multipart/form-data: 여러 부분으로 나뉜 데이터를 하나의 패키지로 전송하는 미디어 유형
- 필요 패키지 설치

```
(.venv)
```

```
$ pip install python-multipart==0.0.9
```

- FastAPI에서 파일 업로드를 처리하기 위해 필요한 의존성 패키지
- multipart/form-data 인코딩을 해석하여 파일 업로드를 가능하게 해줌

파일 업로드

- FastAPI에서는 파일 업로드를 위해 File과 UploadFile이라는 두 가지 주요 클래스를 제공
- File은 파이썬의 내장 file 객체와 유사하고, UploadFile은 추가 메타데이터와 비동기 작업을 위한 메서드를 포함함

파일 업로드

● 예제 코드

... (생략) ...

```
@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile = File(...)):
    # 파일 이름과 콘텐츠 타입 출력
    print("파일 이름:", file.filename)
    print("콘텐츠 타입:", file.content_type)

    # 파일 읽기(비동기)
    contents = await file.read()
    print("파일 크기:", len(contents))

    # 파일 포인터를 처음으로 이동
    file.seek(0)

    # 파일 내용 다시 읽기(비동기)
    contents_again = await file.read()
    print("파일 크기(다시 읽기):", len(contents_again))

    # 파일 리소스 해제
    file.close()

    return {"filename": file.filename}
```

UploadFile 객체

- 파일 업로드 제공 속성과 메서드

- `.filename`: 이 속성은 업로드된 파일의 이름을 문자열로 반환함. 파일 이름을 데이터베이스에 저장하거나 파일을 디스크에 저장할 때 사용함
- `.content_type`: 이 속성은 파일의 콘텐츠 유형(MIME 타입)을 문자열로 반환함. 예를 들어 이미지 파일이면 “image/jpeg”나 “image/png”와 같은 값을 가짐. 이 정보를 통해 파일의 유형을 확인할 수 있음
- `.file`: 이 속성은 파일의 실제 내용을 담고 있는 파일 객체를 반환함. 이 객체를 통해 파일의 내용을 읽거나 다른 처리를 할 수 있음
- `.read()`: 이 메서드는 파일의 내용을 읽어서 반환됨. 이 메서드는 비동기로 작동하므로 `await` 키워드를 사용해야 함
- `.seek()`: 이 메서드는 파일 내의 특정 위치로 포인터를 이동함. 일반적으로 `.read()` 메서드를 사용한 후 다시 파일을 처음부터 읽고 싶을 때 유용함

UploadFile 객체

- 파일을 서버에 저장하기 전 필요한 사전 작업
 - 폴더 생성: 서버가 실행되는 디렉토리에 uploaded_files라는 이름의 폴더 생성. 이 폴더는 업로드된 파일을 저장할 곳임
 - 권한 확인: 이 폴더에 쓰기 권한이 있는지 확인. 권한이 없다면 쓰기 권한 추가

UploadFile 객체

- 예제 코드

```
import shutil
from pathlib import Path
from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile = File(...)):
    folder_name = "uploaded_files"
    Path(folder_name).mkdir(exist_ok=True) # 폴더가 없다면 생성
    file_location = f"{folder_name}/{file.filename}"

    # 읽기/쓰기 포인터를 파일의 시작 위치로 이동
    file.file.seek(0)

    with open(file_location, "wb+") as buffer:
        shutil.copyfileobj(file.file, buffer)

    return {"Info": f"Your file '{file.file.name}' has been uploaded at {file_location}"}
```

UploadFile 객체

● 폴더 생성 확인

- `Path(folder_name).mkdir(exist_ok=True)`는 파이썬의 `pathlib` 라이브러리를 활용해 지정한 폴더(`uploaded_files`)가 없으면 새로 생성함
- `exist_ok=True` 옵션은 해당 폴더가 이미 존재한다면 에러 없이 넘어가라는 의미

● 파일 포인터 초기화

- `file.file.seek(0)` 코드는 파일의 읽기/쓰기 포인터를 파일의 시작점으로 이동시킴
- 이것은 `shutil.copyfileobj()` 함수가 파일의 내용을 처음부터 끝까지 올바르게 복사할 수 있도록 해줌

● 파일 저장

- `with open(file_location, "wb+") as buffer:` 이 부분에서는 업로드된 파일을 서버에 저장
- `open()` 함수의 두 번째 옵션은 파일 모드 옵션: “wb+”는 이진 모드로 파일을 쓰고 읽을 수 있음을 나타냄
- `shutil.copyfileobj(file.file, buffer)`는 파일의 내용을 `buffer`에 복사함